

Documento Técnico-Funcional

Construcción de información del reporte de Reglas de Reabastecimiento en Odoo

Objetivo del documento

Documentar paso a paso cómo se construye la información del reporte **Reordering Rules Detail / Reglas de Reabastecimiento**, explicando cada vista y función involucrada en el cálculo de consumo, cantidad estimada, promedio diario, mínimos, máximos y sugerencia de compra.

El análisis se enfoca especialmente en comprender el origen y comportamiento de la columna **Exp. Qty. (expc_qty)**, así como su relación con las columnas **Sale Qty.**, **Total consumption** y **Daily Average Consumption**.

Flujo general de construcción de información

La información del reporte se construye en el siguiente orden:

1. **explode_lvl_bom_config_lines_pre**
Prepara la información base por producto: venta esperada, fecha de primera venta, existencia, nivel objetivo y venta real de los últimos 90 días.
2. **explode_lvl_bom_config_lines**
Calcula la cantidad esperada (**expc_qty**) y el consumo total (**sum_qty**) a partir de la información preparada.
3. **explode_lvl()**
Explosiona las cantidades por lista de materiales o BOM, llevando la información de productos terminados hacia sus materias primas o componentes.
4. **explode_lvl_bom**
Ordena el resultado de la explosión y calcula el consumo promedio diario (**prom_qty**).
5. **stock_warehouse_orderpoint_report**
Construye el reporte final, agregando proveedor, días de entrega, mínimos, máximos, requeridos y venta abierta.

6. Modelo Odoo `stock.warehouse.orderpoint.report`
Define los campos visibles y campos calculados en Python, como disponibilidad, tránsito, sugerencia y necesidad de compra.
 7. Vista XML del reporte
Define qué columnas se muestran al usuario en pantalla.
-

Paso 1. Vista

`explode_lv1_bom_config_lines_pre`

Nombre de la vista

```
public.explode_lv1_bom_config_lines_pre
```

```
--
```

```
public.explode_lv1_bom_config_lines_pre  
source
```

```
CREATE OR REPLACE VIEW
```

```
public.explode_lv1_bom_config_lines_pre
```

```
AS SELECT sqry.product_id AS id,
```

```
sqry.product_id,
```

```
sqry.date,
```

```
sqry.sale_prc,
```

```
sqry.sale_exp,
```

```
CASE
```

```

        WHEN sqry.stock_offset IS
NULL THEN 0
        ELSE sqry.stock_offset
    END AS stock_offset,
    CASE
        WHEN sqry.qty_available IS
NULL THEN 0::double precision
        ELSE sqry.qty_available
    END AS qty_available,
    CASE
        WHEN sqry.qty_available IS
NULL THEN
        CASE
            WHEN sqry.stock_offset
IS NULL OR sqry.stock_offset < 0 THEN 0
            ELSE sqry.stock_offset
        END::double precision
    ELSE
        CASE
            WHEN sqry.stock_offset
IS NULL OR (sqry.stock_offset::double

```

```

precision - sqry.qty_available) <
0::double precision THEN 0::double
precision

ELSE
sqry.stock_offset::double precision -
sqry.qty_available
END
END AS qty_level,
CASE
WHEN pms90.sum_qty IS NULL
THEN 0::numeric
ELSE pms90.sum_qty -
pms90.desc_qty
END AS sum_qty
FROM ( SELECT pp.id AS product_id,
CASE
WHEN pfs.date < ((
SELECT (( SELECT
timezone('UTC-6'::text,
CURRENT_DATE::timestamp with time zone)

```

```

AS timezone)) - '90
days'::interval))::date THEN 1.00
        ELSE
        CASE
                WHEN pfs.date <
(( SELECT (( SELECT
timezone('UTC-6'::text,
CURRENT_DATE::timestamp with time zone)
AS timezone)) - '60
days'::interval))::date THEN 0.66
        ELSE
        CASE
                WHEN
pfs.date < (( SELECT (( SELECT
timezone('UTC-6'::text,
CURRENT_DATE::timestamp with time zone)
AS timezone)) - '30
days'::interval))::date THEN 0.33
        ELSE 0.00
        END
        END
END

```

```

        END AS sale_prc,
        pt.sale_exp,
        pfs.date,
        pt.qty_for_max_cant::integer
AS stock_offset,
        pas.qty_available,

pt.qty_for_max_cant::integer::double
precision - pas.qty_available AS
qty_level

        FROM product_product pp
        JOIN product_template pt ON
pt.id = pp.product_tmpl_id
        LEFT JOIN product_apr_stock
pas ON pp.id = pas.product_id
        LEFT JOIN
product_first_sale pfs ON pp.id =
pfs.product_id
        WHERE pt.type::text =
'product'::text AND pt.sale_ok IS TRUE
AND pt.can_be_expensed IS FALSE) sqry

```

```
LEFT JOIN product_moves_stats_90
pms90 ON pms90.product_id =
sqry.product_id;
```

Propósito de la vista

Esta vista es el primer paso del cálculo. Su objetivo es preparar la información base de cada producto antes de calcular la cantidad estimada y antes de realizar la explosión de materiales.

En esta etapa todavía no se calcula directamente `expc_qty`; lo que hace esta vista es reunir los datos necesarios para que la siguiente vista pueda calcularlo.

Tablas y vistas que utiliza

La vista toma información principalmente de:

Fuente	Uso dentro de la vista
<code>product_product</code>	Obtiene el producto base.
<code>product_template</code>	Obtiene configuración del producto, como venta esperada y cantidad objetivo.
<code>product_apl_stock</code>	Obtiene existencia disponible.
<code>product_first_sale</code>	Obtiene la fecha de primera venta.
<code>product_moves_stats_90</code>	Obtiene la venta o consumo real de los últimos 90 días.

Filtro principal de productos

La vista considera productos que cumplen estas condiciones:

```
pt.type = 'product'  
AND pt.sale_ok IS TRUE  
AND pt.can_be_expensed IS FALSE
```

Explicación funcional del filtro

Esto significa que la vista sólo considera productos que:

- Son productos almacenables.
- Son productos que se pueden vender.
- No son productos de gasto.

En términos funcionales, esta vista trabaja con productos que pueden formar parte del análisis de venta, consumo y reabastecimiento.

Columnas principales que genera

Columna	Fuente / fórmula	Explicación
<code>product_id</code>	<code>product_product.id</code>	Identifica el producto analizado.
<code>date</code>	<code>product_first_sale.date</code>	Fecha de primera venta del producto.
<code>sale_exp</code>	<code>product_template.sale_exp</code>	Venta esperada configurada en el producto.
<code>sale_prc</code>	Cálculo por antigüedad de primera venta	Porcentaje que representa cuánto historial de venta tiene el producto.
<code>stock_offset</code>	<code>product_template.qty_for_max_cant</code>	Cantidad objetivo o nivel configurado para el producto.
<code>qty_available</code>	<code>product_apt_stock.qty_available</code>	Existencia disponible actual.

<code>qty_level</code>	<code>stock_offset - qty_available</code> , sin negativos	Cantidad faltante para llegar al nivel objetivo.
<code>sum_qty</code>	<code>product_moves_stats_90.sum_qty - product_moves_stats_90.desc_qty</code>	Venta real neta de los últimos 90 días.

Cálculo de `sale_prc`

La columna `sale_prc` se calcula con base en la fecha de primera venta del producto.

```
CASE
  WHEN pfs.date < current_date - interval '90 days' THEN 1.00
  WHEN pfs.date < current_date - interval '60 days' THEN 0.66
  WHEN pfs.date < current_date - interval '30 days' THEN 0.33
  ELSE 0.00
END
```

Explicación funcional de `sale_prc`

`sale_prc` representa el porcentaje de historial de venta que ya tiene el producto.

Antigüedad de la primera venta	Valor de <code>sale_prc</code>	Interpretación
Más de 90 días	<code>1.00</code>	El producto ya tiene historial suficiente.
Más de 60 días y hasta 90 días	<code>0.66</code>	El producto tiene historial parcial.
Más de 30 días y hasta 60 días	<code>0.33</code>	El producto tiene poco historial.
30 días o menos	<code>0.00</code>	El producto es nuevo o casi no tiene historial.

Este valor será utilizado en la siguiente vista para calcular la cantidad esperada (`expc_qty`).

Cálculo de `qty_level`

La columna `qty_level` mide cuánto falta para llegar a la cantidad objetivo del producto.

La lógica funcional es:

`qty_level = stock_offset - qty_available`

Pero la vista protege el cálculo para evitar valores nulos o negativos.

Reglas aplicadas

Caso	Resultado
Si no hay existencia disponible y sí hay cantidad objetivo	Usa <code>stock_offset</code> .
Si hay existencia disponible	Calcula <code>stock_offset - qty_available</code> .
Si el resultado es negativo	Devuelve <code>0</code> .
Si no hay cantidad objetivo	Devuelve <code>0</code> .

Explicación funcional

`qty_level` representa la cantidad faltante para llegar al nivel de inventario configurado.

Ejemplo:

`stock_offset = 1,000`

`qty_available = 300`

`qty_level = 1,000 - 300`

`qty_level = 700`

Si la existencia es mayor que el objetivo:

`stock_offset = 1,000`

`qty_available = 1,200`

`qty_level = 0`

Cálculo de `sum_qty`

La columna `sum_qty` en esta vista representa la venta real neta de los últimos 90 días.

```
CASE
  WHEN pms90.sum_qty IS NULL THEN 0
  ELSE pms90.sum_qty - pms90.desc_qty
END AS sum_qty
```

Explicación funcional

La vista toma la venta o movimiento acumulado de los últimos 90 días y descuenta cantidades que correspondan a devoluciones, descuentos o ajustes, según venga definido en `desc_qty`.

En términos simples:

`sum_qty` = venta real de 90 días - cantidad descontada

Resultado de esta vista

Al finalizar, `explode_lvl_bom_config_lines_pre` deja lista la base para calcular la cantidad estimada.

La salida principal es:

Producto
Fecha de primera venta
Venta esperada
Porcentaje de historial
Cantidad objetivo
Existencia disponible
Cantidad faltante contra objetivo
Venta real neta de últimos 90 días

Explicación textual para negocio

La vista `explode_lvl_bom_config_lines_pre` prepara la información inicial del producto. Identifica productos vendibles y almacenables, obtiene su venta esperada, revisa desde cuándo se venden, consulta su existencia disponible y calcula cuánto han vendido realmente en los últimos 90 días.

También calcula un porcentaje de historial (`sale_prc`) que permite saber si el producto ya tiene suficiente comportamiento real de venta o si todavía necesita apoyarse en una cantidad esperada.

Esta vista no calcula todavía la columna `Exp. Qty.`, pero entrega los datos necesarios para calcularla en el siguiente paso.

Salida hacia la siguiente vista

La siguiente vista, `explode_lvl_bom_config_lines`, toma estos datos y calcula:

```
sale_qty = sum_qty  
expc_qty = sale_exp * (1 - sale_prc)  
sum_qty = sale_qty + expc_qty
```

Es decir, la siguiente vista convierte la información preparada en:

- Venta real.
- Cantidad esperada.
- Consumo total.

Paso 2. Vista

`explode_lvl_bom_config_lines`

Nombre de la vista

```
public.explode_lvl_bom_config_lines
```

```
-- public.explode_lvl_bom_config_lines  
source
```

CREATE OR REPLACE VIEW

```
public.explode_lvl_bom_config_lines
AS SELECT pre.id,
    pre.product_id,
    pre.date,
    pre.sale_exp,
    pre.sale_prc,
    pre.sum_qty AS sale_qty,
    pre.sale_exp::numeric * (1.00 -
pre.sale_prc) AS expc_qty,
    pre.stock_offset,
    pre.qty_available,
    pre.qty_level,
    pre.sum_qty + pre.sale_exp::numeric
* (1.00 - pre.sale_prc) AS sum_qty
    FROM explode_lvl_bom_config_lines_pre
pre;
```

Propósito de la vista

Esta vista toma la información preparada por `explode_lvl_bom_config_lines_pre` y calcula tres valores fundamentales para el reporte:

1. **Venta real** (`sale_qty`)
2. **Cantidad esperada o estimada** (`expc_qty`)
3. **Consumo total** (`sum_qty`)

Esta vista es clave porque aquí se calcula formalmente la columna **Exp. Qty.** (`expc_qty`).

Vista base que utiliza

La vista se construye directamente desde:

FROM `explode_lvl_bom_config_lines_pre` pre

Es decir, recibe como entrada la información ya preparada en el paso anterior:

- Producto.
 - Fecha de primera venta.
 - Venta esperada.
 - Porcentaje de historial.
 - Nivel objetivo.
 - Existencia disponible.
 - Cantidad faltante.
 - Venta real neta de los últimos 90 días.
-

Código principal de la vista

```
CREATE OR REPLACE VIEW public.explode_lvl_bom_config_lines AS
SELECT
  pre.id,
  pre.product_id,
  pre.date,
  pre.sale_exp,
  pre.sale_prc,
  pre.sum_qty AS sale_qty,
  pre.sale_exp::numeric * (1.00 - pre.sale_prc) AS expc_qty,
  pre.stock_offset,
  pre.qty_available,
  pre.qty_level,
```

```
pre.sum_qty + pre.sale_exp::numeric * (1.00 - pre.sale_prc) AS sum_qty
FROM explode_lvl_bom_config_lines_pre pre;
```

Columnas que genera esta vista

Columna	Fórmula / fuente	Explicación
id	pre.id	Identificador del producto.
product_id	pre.product_id	Producto analizado.
date	pre.date	Fecha de primera venta del producto.
sale_exp	pre.sale_exp	Venta esperada configurada en el producto.
sale_prc	pre.sale_prc	Porcentaje de historial calculado por antigüedad de primera venta.
sale_qty	pre.sum_qty	Venta real neta de los últimos 90 días.
expc_qty	pre.sale_exp * (1.00 - pre.sale_prc)	Cantidad esperada o estimada que se agrega al consumo real.
stock_offset	pre.stock_offset	Cantidad objetivo configurada para el producto.
qty_available	pre.qty_available	Existencia disponible.
qty_level	pre.qty_level	Cantidad faltante para llegar al nivel objetivo.
sum_qty	pre.sum_qty + expc_qty	Consumo total: venta real más cantidad esperada.

Cálculo de sale_qty

```
pre.sum_qty AS sale_qty
```

Explicación funcional

`sale_qty` representa la venta real neta de los últimos 90 días.

En el paso anterior, `pre.sum_qty` se calculó como:

venta real acumulada de 90 días - cantidad descontada

Por eso aquí simplemente se renombra como `sale_qty`.

Interpretación

`sale_qty` = venta real neta de últimos 90 días

Cálculo de `expc_qty`

`pre.sale_exp::numeric * (1.00 - pre.sale_prc) AS expc_qty`

Fórmula funcional

$\text{expc_qty} = \text{sale_exp} \times (1 - \text{sale_prc})$

Qué significa

`expc_qty` representa la cantidad esperada o estimada que se agrega al consumo real cuando el producto todavía no tiene suficiente historial de venta.

En otras palabras:

Si el producto es nuevo o tiene poco historial, el sistema no se basa únicamente en la venta real; también considera una parte de la venta esperada configurada en el producto.

Interpretación de `sale_prc` dentro de `expc_qty`

El campo `sale_prc` indica qué porcentaje de historial ya se considera cubierto.

Por lo tanto, la fórmula:

$$\text{expc_qty} = \text{sale_exp} \times (1 - \text{sale_prc})$$

significa:

A la venta esperada se le resta la proporción que ya está cubierta por historial real.

Tabla de comportamiento de **expc_qty**

Antigüedad de primera venta	sale_prc	Porcentaje de sale_exp que se toma como expc_qty
Más de 90 días	1.00	0%
Más de 60 y hasta 90 días	0.66	34%
Más de 30 y hasta 60 días	0.33	67%
30 días o menos	0.00	100%

Ejemplo 1. Producto con más de 90 días de historial

$$\text{sale_exp} = 1,000$$

$$\text{sale_prc} = 1.00$$

$$\text{expc_qty} = 1,000 \times (1 - 1.00)$$

$$\text{expc_qty} = 0$$

Explicación

Como el producto ya tiene más de 90 días de historial, no se agrega cantidad esperada. El sistema considera que la venta real de 90 días ya es suficiente para calcular el consumo.

Ejemplo 2. Producto con historial entre 61 y 90 días

$$\text{sale_exp} = 1,000$$

$$\text{sale_prc} = 0.66$$

$\text{expc_qty} = 1,000 \times (1 - 0.66)$
 $\text{expc_qty} = 340$

Explicación

El producto tiene historial parcial. Por eso se agrega el 34% de la venta esperada.

Ejemplo 3. Producto con historial entre 31 y 60 días

$\text{sale_exp} = 1,000$
 $\text{sale_prc} = 0.33$

$\text{expc_qty} = 1,000 \times (1 - 0.33)$
 $\text{expc_qty} = 670$

Explicación

El producto tiene poco historial. Por eso se agrega el 67% de la venta esperada.

Ejemplo 4. Producto nuevo o con 30 días o menos

$\text{sale_exp} = 1,000$
 $\text{sale_prc} = 0.00$

$\text{expc_qty} = 1,000 \times (1 - 0.00)$
 $\text{expc_qty} = 1,000$

Explicación

Como el producto no tiene historial suficiente, se considera el 100% de la venta esperada.

Cálculo de **sum_qty**

$\text{pre.sum_qty} + \text{pre.sale_exp}::\text{numeric} * (1.00 - \text{pre.sale_prc}) \text{ AS sum_qty}$

Fórmula funcional

$\text{sum_qty} = \text{sale_qty} + \text{expc_qty}$

Qué significa

`sum_qty` representa el consumo total considerado por el reporte.

Está compuesto por:

venta real de últimos 90 días + cantidad esperada

Ejemplo

$\text{sale_qty} = 300$

$\text{expc_qty} = 670$

$\text{sum_qty} = 300 + 670$

$\text{sum_qty} = 970$

Relación entre columnas

La vista genera una relación muy clara:

$\text{sale_qty} = \text{venta real de 90 días}$

$\text{expc_qty} = \text{venta esperada ajustada por historial}$

$\text{sum_qty} = \text{sale_qty} + \text{expc_qty}$

Explicación textual para negocio

La vista `explode_lvl_bom_config_lines` toma la información base del producto y calcula la cantidad esperada que debe complementar la venta real.

Primero identifica la venta real neta de los últimos 90 días y la muestra como `sale_qty`. Después calcula `expc_qty`, que corresponde a la parte de la venta esperada que todavía debe considerarse por falta de historial suficiente. Finalmente suma ambos valores para obtener `sum_qty`, que será el consumo total utilizado en los siguientes cálculos del reporte.

En términos simples, esta vista responde:

¿Cuánto vendí realmente y cuánto debo agregar como estimación para completar el cálculo de consumo?

Importancia de esta vista

Esta vista es crítica porque aquí se define el consumo base del reporte.

A partir de este punto:

sale_qty = venta real
expc_qty = estimación
sum_qty = venta real + estimación

Estos valores se enviarán a la función `explode_lvl()`, donde serán convertidos a nivel de materia prima mediante la explosión de BOM.

Salida hacia la siguiente función

La siguiente pieza del flujo es:

`explode_lvl()`

Esta función tomará los valores:

qty_level
sale_qty
expc_qty
sum_qty

y hará la explosión de materiales.

Es decir, si el producto tiene BOM, convertirá esos valores de producto terminado a sus componentes o materias primas.

Frase clave para memorizar

`explode_lvl_bom_config_lines` convierte la información preparada en tres datos principales:

Venta real = `sale_qty`

Estimación = `expc_qty`

Consumo total = `sale_qty` + `expc_qty`

La columna `expc_qty` es la venta esperada ajustada según el historial real del producto.

Paso 3. Función `explode_lvl()`

Nombre de la función

```
public.explode_lvl()
```

```
-- DROP FUNCTION public.explode_lvl();
```

```
CREATE OR REPLACE FUNCTION
```

```
public.explode_lvl()
```

```
RETURNS TABLE(id integer, product_id
```

```
integer, categ_id integer, uom_id
```

```
integer, qty double precision, sale_qty
```

```
numeric, expc_qty numeric, sum_qty
```

```
numeric)
```

```
LANGUAGE plpgsql
```

AS \$function\$

declare

-- iterar controla el acceso al ciclo

iterar **INTEGER** := **1**;

-- msg TEXT;

begin

-- Esta tabla extrae los
comercializados de las líneas de
configuración

drop table if exists

no_bom_prods;

create temp table no_bom_prods

as

select

pp.id **as** id,

pp.id **as** product_id,

pt.categ_id,

pt.uom_id,

pqsol.qty_level **as**

product_qty,

pqsol.sale_qty,

```
        pqsol.expc_qty,  
        pqsol.sum_qty  
        -- ->PuntoPrueba 1  
from  
  
explode_lv1_bom_config_lines as pqsol  
        inner join product_product pp  
on  
        pp.id = pqsol.product_id  
        inner join product_template pt  
on  
        pp.product_tmpl_id = pt.id  
where  
        pqsol.id not in (  
        select  
            imb.product_id  
        from  
            mrp_bom imb  
        where
```

```
imb.product_id is not  
null and imb.active); ---CAMBIO
```

28/04/2023

-- Esta tabla son los datos principales
del ciclo

```
drop table if exists  
ept_lv1_pnd;  
create temp table ept_lv1_pnd(  
    id INT,  
    product_id INT,  
    categ_id INT,  
    uom_id INT,  
    product_qty numeric,  
    sale_qty numeric,  
    expc_qty numeric,  
    sum_qty numeric  
);
```

insert

into

ept_lv1_pnd

select


```

        elbcl.id,
        elbcl.product_id,
        0,
        0,
        elbcl.qty_level,
        elbcl.sale_qty,
        elbcl.expc_qty,
        elbcl.sum_qty
-- -->PuntoPrueba 1
from

explode_lvl_bom_config_lines as elbcl
where

        elbcl.id not in (

select

        nbp.id

from

        no_bom_prods nbp);
-- Esta tabla alberga el resultado
        drop table if exists
bom_lvl_fnl;

```

```
create temp table bom_lvl_fnl(  
    id INT,  
    product_id INT,  
    categ_id INT,  
    uom_id INT,  
    product_qty numeric,  
    sale_qty numeric,  
    expc_qty numeric,  
    sum_qty numeric  
);
```

```
while iterar>0 loop
```

```
-- Esta tabla extrae los componentes de  
las líneas de configuración
```

```
        drop table if exists  
bom_lvl_all;
```

```
create temp table bom_lvl_all as  
select
```

```
    mbl.product_id as id,  
    mbl.product_id,  
    pt.categ_id,  
    pt.uom_id,
```

```

        SUM(mbl.product_qty *
pqsol.product_qty) as product_qty,
        SUM(mbl.product_qty *
pqsol.sale_qty) as sale_qty,
        SUM(mbl.product_qty *
pqsol.expc_qty) as expc_qty,
        SUM(mbl.product_qty *
pqsol.sum_qty) as sum_qty
-- ->PuntoPrueba 1
from
        ept_lvl_pnd as pqsol
inner join mrp_bom mb on
        mb.product_id =
pqsol.product_id
        and mb.active ---CAMBIO 28/04/2023
inner join mrp_bom_line mbl on
        mbl.bom_id = mb.id
inner join product_product pp on
        pp.id = mbl.product_id
inner join product_template pt on
        pp.product_tmpl_id = pt.id

```

group by

 mbl.product_id,
 pt.categ_id,
 pt.uom_id;

-- Esta tabla extrae de los
componentes, los subproductos

drop table if exists

ept_lv1_pnd;

create temp table ept_lv1_pnd **as**

select

 pp.id,
 pp.id **as** product_id,
 pt.categ_id,
 pt.uom_id,

sum(pmb1.product_qty) **as**

product_qty,

SUM(pmb1.sale_qty) **as**

sale_qty,

SUM(pmb1.expc_qty) **as**

expc_qty,

```

SUM(pmb1.sum_qty) as
sum_qty
from
    bom_lvl_all as pmb1
inner join product_product pp on
    pp.id = pmb1.product_id
inner join product_template pt on
    pp.product_tmpl_id = pt.id
where
    pt.purchase_ok is false
    and pt.sale_ok is false
    and pt.can_be_expensed is false
group by
    pp.id,
    pt.categ_id,
    pt.uom_id;
-- Esta tabla separa las materias
primas de los subproductos
insert
into
bom_lvl_fnl

```

```

                select
                    *
from
    bom_lvl_all
except
                select
                    *
from
    ept_lvl_pnd;
-- Ajustar el número de iteraciones
necesarias
select
    COUNT(ept_lvl_pnd.product_id)
into
    iterar
from
    ept_lvl_pnd;
-- RAISE NOTICE 'Aún debo iterar %
veces', iterar;
end loop;

```

```
-- Devuelve la unión de los  
comercializados, las materias  
-- primas de la explosión
```

```
return query
```

```
select
```

```
    sqry.id,
```

```
    sqry.product_id,
```

```
    sqry.categ_id,
```

```
    sqry.uom_id,
```

```
    sum(sqry.product_qty) as
```

```
product_qty,
```

```
    SUM(sqry.sale_qty) as sale_qty,
```

```
    SUM(sqry.expc_qty) as expc_qty,
```

```
    sum(sqry.sum_qty) as sum_qty
```

```
from
```

```
    (
```

```
        select
```

```
            *
```

```
        from
```

```
            no_bom_prods
```

```
union
```

```
select
    *
from
    bom_lvl_fnl
    ) sqry
group by
    sqry.id,
    sqry.product_id,
    sqry.categ_id,
    sqry.uom_id;
end $function$
;
```

Propósito de la función

La función `explode_lvl()` toma la información generada por la vista `explode_lvl_bom_config_lines` y la transforma a nivel de materia prima o componente.

En otras palabras, esta función responde:

Si tengo venta real, cantidad estimada y consumo total a nivel producto terminado,
¿cuánta materia prima representa eso según la BOM?

Esta función es la encargada de hacer la **explosión de materiales**.

Entrada principal de la función

La función toma como base la vista:

`explode_lvl_bom_config_lines`

De esa vista recibe principalmente estos campos:

Campo recibido	Qué representa
<code>product_id</code>	Producto analizado.
<code>qty_level</code>	Cantidad requerida para nivelación o faltante contra objetivo.
<code>sale_qty</code>	Venta real de los últimos 90 días.
<code>expc_qty</code>	Cantidad esperada o estimada.
<code>sum_qty</code>	Consumo total: venta real + estimación.

Qué devuelve la función

La función devuelve una tabla con estas columnas:

```
RETURNS TABLE(  
  id integer,  
  product_id integer,  
  categ_id integer,  
  uom_id integer,  
  qty double precision,  
  sale_qty numeric,  
  expc_qty numeric,  
  sum_qty numeric  
)
```

Explicación de columnas devueltas

Columna	Qué representa
<code>id</code>	Identificador del producto o componente final.
<code>product_id</code>	Producto o materia prima resultante.
<code>categ_id</code>	Categoría del producto.
<code>uom_id</code>	Unidad de medida.
<code>qty</code>	Cantidad requerida por nivelación.
<code>sale_qty</code>	Venta real convertida a nivel componente.
<code>expc_qty</code>	Cantidad esperada convertida a nivel componente.
<code>sum_qty</code>	Consumo total convertido a nivel componente.

Lógica general de la función

La función trabaja en dos caminos:

1. Productos sin BOM activa
→ pasan directo al resultado.
2. Productos con BOM activa
→ se explodian a sus componentes.

Además, si encuentra componentes intermedios, los vuelve a explotar hasta llegar a los componentes finales.

1. Identificación de productos sin BOM

La primera parte de la función crea una tabla temporal llamada:

`no_bom_prods`

Código principal:

```
CREATE TEMP TABLE no_bom_prods AS
SELECT
  pp.id AS id,
  pp.id AS product_id,
  pt.categ_id,
  pt.uom_id,
  pqsol.qty_level AS product_qty,
  pqsol.sale_qty,
  pqsol.expc_qty,
  pqsol.sum_qty
FROM explode_lvl_bom_config_lines AS pqsol
INNER JOIN product_product pp ON pp.id = pqsol.product_id
INNER JOIN product_template pt ON pp.product_tmpl_id = pt.id
WHERE pqsol.id NOT IN (
  SELECT imb.product_id
  FROM mrp_bom imb
  WHERE imb.product_id IS NOT NULL
  AND imb.active
);
```

Qué hace esta parte

Busca productos de `explode_lvl_bom_config_lines` que **no tienen una BOM activa**.

Si un producto no tiene BOM activa, la función no puede explosionarlo. Por eso lo deja como producto final.

Explicación funcional

Si el producto no tiene lista de materiales, se conserva tal cual.

Ejemplo:

Producto A no tiene BOM.

Entonces:

sale_qty = pasa igual
expc_qty = pasa igual
sum_qty = pasa igual
qty_level = pasa igual

Esto aplica para productos comercializados o productos que no requieren descomposición en componentes.

2. Identificación de productos con BOM

Después la función crea otra tabla temporal:

ept_lvl_pnd

Código principal:

```
CREATE TEMP TABLE ept_lvl_pnd(  
  id INT,  
  product_id INT,  
  categ_id INT,  
  uom_id INT,  
  product_qty numeric,  
  sale_qty numeric,  
  expc_qty numeric,  
  sum_qty numeric  
);  
  
INSERT INTO ept_lvl_pnd  
SELECT  
  elbcl.id,  
  elbcl.product_id,  
  0,  
  0,  
  elbcl.qty_level,  
  elbcl.sale_qty,  
  elbcl.expc_qty,  
  elbcl.sum_qty  
FROM explode_lvl_bom_config_lines AS elbcl  
WHERE elbcl.id NOT IN (  
  SELECT nbp.id  
  FROM no_bom_prods nbp  
);
```

Qué hace esta parte

Toma los productos que **sí tienen BOM activa** y los coloca en una tabla temporal de pendientes por explotar.

Explicación funcional

Aquí se separan los productos que deben transformarse en componentes.

Ejemplo:

Producto terminado A sí tiene BOM.

Entonces no se queda como A.

Se manda a ept_lvl_pnd para explotarlo hacia sus materias primas.

3. Tabla final de resultados de BOM

La función crea una tabla temporal llamada:

bom_lvl_fnl

Código:

```
CREATE TEMP TABLE bom_lvl_fnl(  
  id INT,  
  product_id INT,  
  categ_id INT,  
  uom_id INT,  
  product_qty numeric,  
  sale_qty numeric,  
  expc_qty numeric,  
  sum_qty numeric  
);
```

Qué hace esta tabla

Esta tabla guarda los componentes finales que resultan de la explosión de BOM.

En otras palabras:

Aquí se van acumulando las materias primas o productos finales que ya no deben seguir explosionándose.

4. Ciclo de explosión de BOM

La función usa un ciclo:

WHILE iterar > 0 LOOP

Este ciclo se repite mientras existan productos intermedios pendientes de explosionar.

4.1 Tabla **bom_lvl_all**

Dentro del ciclo, se crea:

bom_lvl_all

Código principal:

```
CREATE TEMP TABLE bom_lvl_all AS
SELECT
  mbl.product_id AS id,
  mbl.product_id,
  pt.categ_id,
  pt.uom_id,
  SUM(mbl.product_qty * pqsol.product_qty) AS product_qty,
  SUM(mbl.product_qty * pqsol.sale_qty) AS sale_qty,
  SUM(mbl.product_qty * pqsol.expc_qty) AS expc_qty,
  SUM(mbl.product_qty * pqsol.sum_qty) AS sum_qty
FROM ept_lvl_pnd AS pqsol
INNER JOIN mrp_bom mb ON mb.product_id = pqsol.product_id
  AND mb.active
INNER JOIN mrp_bom_line mbl ON mbl.bom_id = mb.id
INNER JOIN product_product pp ON pp.id = mbl.product_id
INNER JOIN product_template pt ON pp.product_tmpl_id = pt.id
GROUP BY
  mbl.product_id,
  pt.categ_id,
  pt.uom_id;
```

Qué hace esta parte

Esta es la explosión de la BOM.

Por cada producto pendiente, busca su BOM activa y obtiene sus componentes.

Luego multiplica las cantidades del producto padre por la cantidad requerida del componente en la BOM.

Fórmulas principales de la explosión

Cantidad requerida por nivelación

product_qty del componente =
cantidad BOM del componente × qty_level del producto padre

Venta real

sale_qty del componente =
cantidad BOM del componente × sale_qty del producto padre

Cantidad estimada

expc_qty del componente =
cantidad BOM del componente × expc_qty del producto padre

Consumo total

sum_qty del componente =
cantidad BOM del componente × sum_qty del producto padre

Ejemplo práctico

Producto terminado A:

qty_level = 100

sale_qty = 300
expc_qty = 700
sum_qty = 1,000

BOM de producto A:

Materia Prima X = 2 unidades por cada A
Materia Prima Y = 5 unidades por cada A

Resultado para Materia Prima X:

qty_level = $100 \times 2 = 200$
sale_qty = $300 \times 2 = 600$
expc_qty = $700 \times 2 = 1,400$
sum_qty = $1,000 \times 2 = 2,000$

Resultado para Materia Prima Y:

qty_level = $100 \times 5 = 500$
sale_qty = $300 \times 5 = 1,500$
expc_qty = $700 \times 5 = 3,500$
sum_qty = $1,000 \times 5 = 5,000$

5. Identificación de productos intermedios

Después de explotar la BOM, la función identifica componentes que son productos intermedios.

Código:

```
CREATE TEMP TABLE ept_lvl_pnd AS
SELECT
  pp.id,
  pp.id AS product_id,
  pt.categ_id,
  pt.uom_id,
  SUM(pmbi.product_qty) AS product_qty,
  SUM(pmbi.sale_qty) AS sale_qty,
  SUM(pmbi.expc_qty) AS expc_qty,
  SUM(pmbi.sum_qty) AS sum_qty
FROM bom_lvl_all AS pmbi
INNER JOIN product_product pp ON pp.id = pmbi.product_id
INNER JOIN product_template pt ON pp.product_tmpl_id = pt.id
```



```
WHERE pt.purchase_ok IS FALSE
AND pt.sale_ok IS FALSE
AND pt.can_be_expensed IS FALSE
GROUP BY
    pp.id,
    pt.categ_id,
    pt.uom_id;
```

Qué considera como producto intermedio

Un producto se considera intermedio cuando:

```
purchase_ok = False
sale_ok = False
can_be_expensed = False
```

Explicación funcional

Esto significa que el componente:

- No se compra directamente.
- No se vende.
- No es gasto.

Por lo tanto, probablemente es un subensamble o componente intermedio que también puede tener BOM.

6. Separación de materias primas finales

Después, la función inserta en `bom_lvl_fnl` todo lo que salió de `bom_lvl_all`, excepto los productos intermedios:

```
INSERT INTO bom_lvl_fnl
SELECT * FROM bom_lvl_all
EXCEPT
SELECT * FROM ept_lvl_pnd;
```

Qué hace

Separa los componentes finales de los productos intermedios.

Componentes finales → se guardan en bom_lvl_fnl.

Productos intermedios → regresan a ept_lvl_pnd para explosionarse otra vez.

7. Control del ciclo

La función cuenta cuántos productos intermedios quedaron pendientes:

```
SELECT COUNT(ept_lvl_pnd.product_id)
INTO iterar
FROM ept_lvl_pnd;
```

Qué significa

Si todavía hay productos intermedios:

iterar > 0

el ciclo vuelve a correr.

Si ya no hay productos intermedios:

iterar = 0

la función termina la explosión.

8. Resultado final de la función

Al final, la función une:

```
no_bom_prods
UNION
bom_lvl_fnl
```

Código:

```
RETURN QUERY
SELECT
```

```

    sqry.id,
    sqry.product_id,
    sqry.categ_id,
    sqry.uom_id,
    SUM(sqry.product_qty) AS product_qty,
    SUM(sqry.sale_qty) AS sale_qty,
    SUM(sqry.expc_qty) AS expc_qty,
    SUM(sqry.sum_qty) AS sum_qty
FROM (
    SELECT * FROM no_bom_prods
    UNION
    SELECT * FROM bom_lvl_fnl
) sqry
GROUP BY
    sqry.id,
    sqry.product_id,
    sqry.categ_id,
    sqry.uom_id;

```

Qué hace esta salida

Consolida todo en una sola tabla final:

Origen	Qué contiene
<code>no_bom_prods</code>	Productos sin BOM activa que pasan directo.
<code>bom_lvl_fnl</code>	Componentes finales resultantes de la explosión de BOM.

Después agrupa por producto para evitar duplicados.

Resultado funcional final

La función devuelve productos o materias primas finales con sus cantidades consolidadas:

Campo	Significado final
-------	-------------------

<code>product_qty / qty</code>	Cantidad requerida por nivelación.
<code>sale_qty</code>	Venta real convertida a nivel producto/componente final.
<code>expc_qty</code>	Cantidad estimada convertida a nivel producto/componente final.
<code>sum_qty</code>	Consumo total convertido a nivel producto/componente final.

Explicación textual para negocio

La función `explode_lvl()` toma la venta real, la cantidad esperada y el consumo total calculados previamente y los convierte a nivel de materia prima mediante la lista de materiales.

Si el producto no tiene BOM, conserva sus cantidades tal como vienen. Si el producto tiene BOM, multiplica sus cantidades por los componentes definidos en la lista de materiales. Si encuentra productos intermedios, repite el proceso hasta llegar a componentes finales.

Esto permite que el reporte no sólo muestre consumo a nivel producto terminado, sino también la cantidad real y estimada de materias primas necesarias para cubrir ese consumo.

Importancia de `expc_qty` dentro de esta función

Antes de entrar a esta función, `expc_qty` ya está calculado como:

$$\text{expc_qty} = \text{sale_exp} \times (1 - \text{sale_prc})$$

Dentro de `explode_lvl()`, esa cantidad se transforma así:

`expc_qty` del componente =
`expc_qty` del producto padre $\times$ cantidad del componente en BOM

Por lo tanto, `expc_qty` deja de ser una estimación a nivel producto terminado y se convierte en una estimación a nivel materia prima.

Frase clave para memorizar

`explode_lvl()` es la función que baja los cálculos de venta real, estimación y consumo total desde producto terminado hacia materia prima usando la BOM.

En una frase:

Producto terminado vendido o estimado

× cantidad de componente en BOM

= consumo real o estimado de materia prima

Paso 4. Vista `explode_lvl_bom`

Nombre de la vista

`public.explode_lvl_bom`

Propósito de la vista

La vista `explode_lvl_bom` toma el resultado de la función `explode_lvl()` y lo prepara para el reporte final.

Su principal función es:

1. Asociar el resultado de la explosión con información del producto.
2. Traer clasificación, unidad de medida y tipo de empaque.
3. Calcular el **consumo promedio diario** (`prom_qty`).
4. Dejar lista la información que después será usada por `stock_warehouse_orderpoint_report`.

En esta etapa, la información ya viene convertida a nivel de producto final o materia prima, dependiendo de si pasó o no por BOM.

Código de la vista

```
CREATE OR REPLACE VIEW public.explode_lvl_bom AS
SELECT
  pt.id AS product_tmpl_id,
  elvl.product_id,
  elvl.categ_id,
  pt.purchase_clasif,
  elvl.uom_id,
  pt.package_type,
  elvl.qty AS qty_req,
  elvl.sale_qty,
  elvl.expc_qty,
  elvl.sum_qty,
  elvl.sum_qty / 90::numeric AS prom_qty
FROM explode_lvl() elvl(
  id,
  product_id,
  categ_id,
  uom_id,
  qty,
  sale_qty,
  expc_qty,
  sum_qty
)
JOIN product_product pp
  ON elvl.product_id = pp.id
JOIN product_template pt
  ON pp.product_tmpl_id = pt.id;
```

Vista o función que utiliza

Esta vista consume directamente la función:

explode_lvl()

Es decir, recibe los valores ya procesados por la explosión de BOM:

Campo recibido de <code>explode_lvl()</code>	Qué representa
<code>product_id</code>	Producto o materia prima resultante.
<code>categ_id</code>	Categoría del producto.
<code>uom_id</code>	Unidad de medida.
<code>qty</code>	Cantidad requerida por nivelación.
<code>sale_qty</code>	Venta real convertida a nivel producto/componente.
<code>expc_qty</code>	Cantidad estimada convertida a nivel producto/componente.
<code>sum_qty</code>	Consumo total convertido a nivel producto/componente.

Tablas adicionales que cruza

Además de `explode_lvl()`, cruza con:

`product_product` pp
`product_template` pt

Para qué las usa

Tabla	Uso
<code>product_product</code>	Relaciona el producto resultante de la explosión con su plantilla.
<code>product_template</code>	Obtiene información funcional del producto, como clasificación de compra, tipo de empaque y plantilla del producto.

Columnas que genera esta vista

Columna	Fuente / fórmula	Explicación
<code>product_tmpl_id</code>	<code>pt.id</code>	Identificador de la plantilla del producto.
<code>product_id</code>	<code>elvl.product_id</code>	Producto o materia prima resultante.
<code>categ_id</code>	<code>elvl.categ_id</code>	Categoría del producto.
<code>purchase_classification</code>	<code>pt.purchase_classification</code>	Clasificación de compra del producto.
<code>uom_id</code>	<code>elvl.uom_id</code>	Unidad de medida.
<code>package_type</code>	<code>pt.package_type</code>	Tipo de empaque del producto.
<code>qty_req</code>	<code>elvl.qty</code>	Cantidad requerida por nivelación.
<code>sale_qty</code>	<code>elvl.sale_qty</code>	Venta real convertida al nivel del producto o componente final.
<code>expc_qty</code>	<code>elvl.expc_qty</code>	Cantidad estimada convertida al nivel del producto o componente final.
<code>sum_qty</code>	<code>elvl.sum_qty</code>	Consumo total: venta real más estimación.
<code>prom_qty</code>	<code>elvl.sum_qty / 90</code>	Consumo promedio diario.

Cálculo de `qty_req`

`elvl.qty` AS `qty_req`

Qué significa

`qty_req` representa la cantidad requerida por nivelación.

Este valor viene desde `qty_level`, que previamente se calculó como el faltante contra el nivel objetivo del producto.

Cuando el producto pasó por BOM, ese faltante también fue explotado hacia sus componentes.

Explicación funcional

`qty_req` responde a la pregunta:

¿Cuánto necesito para completar el nivel objetivo de inventario?

Cálculo de `sale_qty`

`elvl.sale_qty`

Qué significa

`sale_qty` representa la venta real de los últimos 90 días, pero ya transformada al nivel del producto o materia prima que aparece en el reporte.

Si el producto no tenía BOM, pasa directo.

Si tenía BOM, se multiplica por la cantidad de componente requerida en la lista de materiales.

Explicación funcional

`sale_qty` responde a la pregunta:

¿Cuánto se consumió realmente de este material por ventas de los últimos 90 días?

Cálculo de `expc_qty`

`elvl.expc_qty`

Qué significa

expc_qty representa la cantidad estimada esperada, ya convertida al nivel del producto o materia prima.

Antes de llegar aquí, se calculó como:

$$\text{expc_qty} = \text{sale_exp} \times (1 - \text{sale_prc})$$

Y si el producto tenía BOM, se transformó así:

$$\text{expc_qty componente} = \text{expc_qty producto padre} \times \text{cantidad del componente en BOM}$$

Explicación funcional

expc_qty responde a la pregunta:

¿Cuánta cantidad adicional debo considerar como estimación para este material?

Cálculo de **sum_qty**

lvl.sum_qty

Qué significa

sum_qty representa el consumo total.

Su lógica base es:

$$\text{sum_qty} = \text{sale_qty} + \text{expc_qty}$$

Cuando el producto pasa por BOM, ese consumo total también se convierte a nivel materia prima.

Explicación funcional

sum_qty responde a la pregunta:

¿Cuál es el consumo total considerado para este material, sumando venta real más estimación?

Cálculo de **prom_qty**

elvl.sum_qty / 90::numeric AS prom_qty

Fórmula funcional

$\text{prom_qty} = \text{sum_qty} / 90$

Qué significa

prom_qty representa el **consumo promedio diario**.

Divide el consumo total entre 90 días.

Explicación funcional

prom_qty responde a la pregunta:

¿Cuánto se consume en promedio por día de este material?

Ejemplo práctico

Supongamos que después de la explosión de BOM tenemos una materia prima con estos valores:

sale_qty = 600
expc_qty = 1,400
sum_qty = 2,000

Entonces:

$\text{prom_qty} = \text{sum_qty} / 90$
 $\text{prom_qty} = 2,000 / 90$
 $\text{prom_qty} = 22.22$

Interpretación

El sistema considera que de esa materia prima se consumen en promedio **22.22 unidades por día**.

Relación entre columnas

La relación principal de esta vista es:

$\text{sale_qty} + \text{expc_qty} = \text{sum_qty}$

$\text{sum_qty} / 90 = \text{prom_qty}$

Visualmente:

Venta real

+

Cantidad estimada

=

Consumo total

÷

90 días

=

Consumo promedio diario

Explicación textual para negocio

La vista `explode_lvl_bom` recibe la información ya explosionada a nivel de materia prima o componente. A partir de esa información, identifica el producto, su clasificación de compra, unidad de medida y tipo de empaque.

También calcula el consumo promedio diario dividiendo el consumo total entre 90 días. Este promedio será utilizado posteriormente para calcular cuánto material se necesita durante el tiempo de entrega del proveedor.

En términos funcionales, esta vista convierte el consumo total en una tasa diaria de consumo.

Importancia de esta vista

Esta vista es importante porque entrega el dato base para los cálculos de reabastecimiento:

$\text{prom_qty} = \text{consumo promedio diario}$

A partir de **prom_qty**, la siguiente vista podrá calcular:

$\text{delay_x_cpd} = \text{días de entrega} \times \text{consumo promedio diario}$

Ese cálculo será la base para mínimos, máximos y sugerencia de compra.

Salida hacia la siguiente vista

La siguiente pieza es:

stock_warehouse_orderpoint_report

Esta vista tomará la información de **explode_lvl_bom** y la cruzará con proveedor para calcular:

- Proveedor.
 - Días de entrega.
 - Mínimo proveedor.
 - Consumo durante el tiempo de entrega.
 - Mínimo calculado.
 - Máximo calculado.
 - Cantidad requerida.
 - Venta abierta.
-

Frase clave para memorizar

explode_lvl_bom toma el consumo total ya explosionado y lo convierte en consumo promedio diario.

En una frase:

$\text{sum_qty} / 90 = \text{prom_qty}$

Es decir:

Consumo total de 90 días $\div$ 90 = consumo diario promedio

Paso 5. Vista

stock_warehouse_orderpoint_report

Nombre de la vista

```
public.stock_warehouse_orderpoint_report
```

```
--
```

```
public.stock_warehouse_orderpoint_report source
```

```
CREATE OR REPLACE VIEW
```

```
public.stock_warehouse_orderpoint_report
```

```
AS SELECT qry.product_id AS id,
```

```
    qry.product_id,
```

```
    qry.purchase_clasif AS categ_id,
```

```
    qry.uom_id,
```

```
    qry.package_type,
```

```
    qry.sale_qty,
```

```
    qry.expc_qty,
```

```

qry.sum_qty,
qry.prom_qty,
qry.supplier_info_id,
qry.delay,
qry.delay_x_cpd,
qry.min_qty,
qry.res_max,
qry.res_min,
CASE
    WHEN qry.qty_req IS NULL
THEN 0::double precision
    ELSE qry.qty_req
END AS qty_req,
CASE
    WHEN esd.qty IS NULL THEN
0::numeric
    ELSE esd.qty
END AS qty_sale,
false AS need_purchased,
max(ppp.x_msg_exp) AS x_msg_exp
FROM ( SELECT smls.product_tmpl_id,

```

```

        smls.product_id,
        smls.categ_id,
        smls.uom_id,
        smls.package_type,
        smls.purchase_clasif,
        smls.sale_qty,
        smls.expc_qty,
        smls.sum_qty,
        smls.prom_qty,
        (array_agg(ps.id))[1] AS
supplier_info_id,
        (array_agg(ps.delay))[1] AS
delay,
        (array_agg(ps.delay::numeric
* smls.prom_qty))[1] AS delay_x_cpd,
        (array_agg(ps.min_qty))[1]
AS min_qty,
        CASE
            WHEN
pt.product_consigna = true THEN

```



```

(array_agg(ps.delay::numeric *
smls.prom_qty))[1]
                ELSE
(array_agg(ps.delay::numeric *
smls.prom_qty * 2::numeric))[1]
                END AS res_min,
        (array_agg(
                CASE
                        WHEN
(ps.delay::numeric *
smls.prom_qty)::double precision >
ps.min_qty THEN (ps.delay::numeric *
smls.prom_qty * 2::numeric)::double
precision
                        ELSE
(ps.delay::numeric *
smls.prom_qty)::double precision +
ps.min_qty
                END))[1] AS res_max,
        smls.qty_req
FROM explode_lvl_bom smls

```

JOIN product_template pt **ON**
pt.id = smls.product_tmpl_id

JOIN product_supplierinfo
ps **ON** ps.product_tmpl_id =
smls.product_tmpl_id

GROUP BY smls.product_tmpl_id,
smls.product_id, pt.product_consigna,
smls.categ_id, smls.uom_id,
smls.package_type,
smls.purchase_clasif, smls.sale_qty,
smls.expc_qty, smls.sum_qty,
smls.prom_qty, smls.qty_req) qry

LEFT JOIN explode_so_days('30
days'::interval) esd(id, product_id,
categ_id, uom_id, qty) **ON**
esd.product_id = qry.product_id

LEFT JOIN product_product ppp **ON**
ppp.id = qry.product_id

GROUP BY qry.product_id, qry.uom_id,
qry.package_type, qry.purchase_clasif,
qry.sale_qty, qry.expc_qty,

```
qry.sum_qty, qry.prom_qty,  
qry.supplier_info_id, qry.delay,  
qry.delay_x_cpd, qry.min_qty,  
qry.res_max, qry.res_min, qry.qty_req,  
esd.qty;
```

Propósito de la vista

La vista `stock_warehouse_orderpoint_report` construye la información final del reporte de reglas de reabastecimiento.

Toma los datos ya preparados por `explode_lvl_bom`, principalmente:

- Producto.
- Categoría.
- Unidad de medida.
- Tipo de empaque.
- Venta real.
- Cantidad esperada.
- Consumo total.
- Consumo promedio diario.
- Cantidad requerida por nivelación.

Después cruza esa información con datos del proveedor para calcular:

- Proveedor.
- Días de entrega.
- Consumo durante el tiempo de entrega.
- Cantidad mínima proveedor.
- Mínimo.
- Máximo.
- Cantidad requerida.
- Venta abierta de próximos días.

Esta vista ya representa la base principal del reporte final en Odoo.

Código base de la vista

```
CREATE OR REPLACE VIEW public.stock_warehouse_orderpoint_report AS
SELECT
    qry.product_id AS id,
    qry.product_id,
    qry.purchase_clasif AS categ_id,
    qry.uom_id,
    qry.package_type,
    qry.sale_qty,
    qry.expc_qty,
    qry.sum_qty,
    qry.prom_qty,
    qry.supplier_info_id,
    qry.delay,
    qry.delay_x_cpd,
    qry.min_qty,
    qry.res_max,
    qry.res_min,
    CASE
        WHEN qry.qty_req IS NULL THEN 0
        ELSE qry.qty_req
    END AS qty_req,
    CASE
        WHEN esd.qty IS NULL THEN 0
        ELSE esd.qty
    END AS qty_sale,
    false AS need_purchased,
    max(ppp.x_msg_exp) AS x_msg_exp
FROM (...)
LEFT JOIN explode_so_days('30 days'::interval) esd
    ON esd.product_id = qry.product_id
LEFT JOIN product_product ppp
    ON ppp.id = qry.product_id
GROUP BY ...
```

Vista principal que utiliza

La vista se alimenta principalmente de:

`explode_lvl_bom` `smls`

`explode_lvl_bom` ya trae:

Campo	Qué representa
<code>product_tmpl_id</code>	Plantilla del producto.
<code>product_id</code>	Producto o materia prima resultante.
<code>categ_id</code>	Categoría.
<code>purchase_classification</code>	Clasificación de compra.
<code>uom_id</code>	Unidad de medida.
<code>package_type</code>	Tipo de empaque.
<code>qty_req</code>	Cantidad requerida por nivelación.
<code>sale_qty</code>	Venta real.
<code>expc_qty</code>	Cantidad esperada.
<code>sum_qty</code>	Consumo total.
<code>prom_qty</code>	Consumo promedio diario.

Cruce con proveedor

Dentro de la subconsulta `qry`, la vista cruza con:

```
JOIN product_supplierinfo ps
  ON ps.product_tmpl_id = smls.product_tmpl_id
```

Qué significa

Para cada producto, busca su información de proveedor en `product_supplierinfo`.

De ahí toma:

Campo	Fuente	Qué representa
<code>supplier_info_id</code>	<code>ps.id</code>	Registro de proveedor del producto.
<code>delay</code>	<code>ps.delay</code>	Días de entrega del proveedor.
<code>min_qty</code>	<code>ps.min_qty</code>	Cantidad mínima del proveedor.

Subconsulta principal `qry`

La vista final está construida sobre una subconsulta llamada `qry`.

Esta subconsulta calcula los principales valores de reabastecimiento:

```
SELECT
  smls.product_tmpl_id,
  smls.product_id,
  smls.categ_id,
  smls.uom_id,
  smls.package_type,
  smls.purchase_clasif,
  smls.sale_qty,
  smls.expc_qty,
  smls.sum_qty,
  smls.prom_qty,
  (array_agg(ps.id))[1] AS supplier_info_id,
  (array_agg(ps.delay))[1] AS delay,
  (array_agg(ps.delay::numeric * smls.prom_qty))[1] AS delay_x_cpd,
  (array_agg(ps.min_qty))[1] AS min_qty,
  CASE
    WHEN pt.product_consigna = true
      THEN (array_agg(ps.delay::numeric * smls.prom_qty))[1]
    ELSE
      (array_agg(ps.delay::numeric * smls.prom_qty * 2))[1]
  END AS res_min,
```

```
(array_agg(
  CASE
    WHEN (ps.delay::numeric * smls.prom_qty)::double precision > ps.min_qty
    THEN (ps.delay::numeric * smls.prom_qty * 2)::double precision
    ELSE (ps.delay::numeric * smls.prom_qty)::double precision + ps.min_qty
  END
))[1] AS res_max,
smls.qty_req
FROM explode_lvl_bom smls
JOIN product_template pt
  ON pt.id = smls.product_tmpl_id
JOIN product_supplierinfo ps
  ON ps.product_tmpl_id = smls.product_tmpl_id
GROUP BY ...
```

Uso de `array_agg( . . . ) [ 1 ]`

La vista usa varias veces:

```
(array_agg(campo))[1]
```

Qué significa

Agrupar los datos del proveedor y tomar el primer valor del arreglo.

Ejemplo:

```
(array_agg(ps.delay))[1] AS delay
```

Explicación funcional

Si el producto tiene más de un proveedor en `product_supplierinfo`, la vista toma solamente uno.

Observación importante

No hay un `ORDER BY` dentro del `array_agg`, por lo tanto, técnicamente, el proveedor seleccionado puede depender del orden que devuelva PostgreSQL.

Para una selección totalmente controlada, debería existir una regla de ordenamiento, por ejemplo:

```
array_agg(ps.delay ORDER BY ps.delay ASC)
```

o una priorización por menor entrega, menor costo, frecuencia, etc.

Columnas finales que genera la vista

Columna	Fuente / fórmula	Explicación
<code>id</code>	<code>qry.product_id</code>	Identificador del registro en el reporte.
<code>product_id</code>	<code>qry.product_id</code>	Producto o materia prima.
<code>categ_id</code>	<code>qry.purchase_clasif</code>	Clasificación de compra.
<code>uom_id</code>	<code>qry.uom_id</code>	Unidad de medida.
<code>package_type</code>	<code>qry.package_type</code>	Tipo de empaque.
<code>sale_qty</code>	<code>qry.sale_qty</code>	Venta real de últimos 90 días, ya procesada.
<code>expc_qty</code>	<code>qry.expc_qty</code>	Cantidad esperada ya procesada.
<code>sum_qty</code>	<code>qry.sum_qty</code>	Consumo total: venta real más estimación.
<code>prom_qty</code>	<code>qry.prom_qty</code>	Consumo promedio diario.
<code>supplier_info_id</code>	Primer <code>ps.id</code> agrupado	Proveedor seleccionado.
<code>delay</code>	Primer <code>ps.delay</code> agrupado	Días de entrega del proveedor.
<code>delay_x_cpd</code>	<code>delay * prom_qty</code>	Consumo durante el tiempo de entrega.
<code>min_qty</code>	Primer <code>ps.min_qty</code> agrupado	Cantidad mínima proveedor.
<code>res_min</code>	Depende de consigna	Mínimo calculado.

<code>res_max</code>	Comparación entre <code>delay_x_cpd</code> y <code>min_qty</code>	Máximo calculado.
<code>qty_req</code>	<code>qry.qty_req</code> , si es nulo 0	Cantidad requerida por nivelación.
<code>qty_sale</code>	<code>esd.qty</code> , si es nulo 0	Venta abierta / requerimiento por órdenes próximas.
<code>need_purchase</code> <code>d</code>	<code>false</code>	Campo inicializado en falso; después se ajusta en Python.
<code>x_msg_exp</code>	<code>max(ppp.x_msg_exp)</code>	Mensaje o dato adicional del producto.

Cálculo de `delay`

`(array_agg(ps.delay))[1]` AS delay

Qué significa

`delay` representa los días de entrega del proveedor.

Explicación funcional

Es el tiempo estimado que tarda el proveedor en surtir el producto.

Ejemplo:

delay = 15 días

Cálculo de `delay_x_cpd`

`(array_agg(ps.delay::numeric * smls.prom_qty))[1]` AS delay_x_cpd

Fórmula funcional

`delay_x_cpd = delay * prom_qty`

Qué significa

Representa el consumo esperado durante los días de entrega del proveedor.

Ejemplo

prom_qty = 100 piezas por día
delay = 15 días

delay_x_cpd = 15 * 100
delay_x_cpd = 1,500

Interpretación

Si el proveedor tarda 15 días, y se consumen 100 piezas diarias, se necesitan 1,500 piezas para cubrir ese periodo de espera.

Cálculo de **min_qty**

```
(array_agg(ps.min_qty))[1] AS min_qty
```

Qué significa

min_qty representa la cantidad mínima de compra del proveedor.

Explicación funcional

Es el mínimo que el proveedor permite comprar.

Ejemplo:

Aunque yo necesite 800 piezas,
si el proveedor tiene mínimo 1,000,
min_qty = 1,000

Cálculo de **res_min**

```
CASE
  WHEN pt.product_consigna = true
    THEN (array_agg(ps.delay::numeric * smls.prom_qty))[1]
  ELSE
    (array_agg(ps.delay::numeric * smls.prom_qty * 2))[1]
END AS res_min
```

Fórmula funcional

Si product_consigna = true:
 $\text{res_min} = \text{delay} * \text{prom_qty}$

Si product_consigna = false:
 $\text{res_min} = \text{delay} * \text{prom_qty} * 2$

Qué significa

`res_min` representa el mínimo calculado del reporte.

Pero su fórmula cambia dependiendo de si el producto es de consigna.

Caso 1. Producto de consigna

$\text{res_min} = \text{delay} * \text{prom_qty}$

Interpretación

Para productos de consigna, el mínimo se calcula como el consumo necesario para cubrir únicamente el tiempo de entrega.

Caso 2. Producto normal

$\text{res_min} = \text{delay} * \text{prom_qty} * 2$

Interpretación

Para productos normales, el mínimo se duplica como margen de seguridad.

Cálculo de **res_max**

```
CASE
  WHEN delay * prom_qty > min_qty
    THEN delay * prom_qty * 2
  ELSE delay * prom_qty + min_qty
END
```

Fórmula funcional

Si $\text{delay_x_cpd} > \text{min_qty}$:
 $\text{res_max} = \text{delay_x_cpd} * 2$

Si $\text{delay_x_cpd} \leq \text{min_qty}$:
 $\text{res_max} = \text{delay_x_cpd} + \text{min_qty}$

Qué significa

res_max representa el máximo calculado para reabastecimiento.

Compara el consumo durante el tiempo de entrega contra la cantidad mínima del proveedor.

Caso 1. El consumo durante entrega supera el mínimo proveedor

$\text{delay_x_cpd} = 1,500$
 $\text{min_qty} = 1,000$

Como $1,500 > 1,000$:

$\text{res_max} = 1,500 * 2$
 $\text{res_max} = 3,000$

Interpretación

Si la operación consume más que el mínimo del proveedor durante el tiempo de entrega, el máximo se calcula como el doble de ese consumo.

Caso 2. El mínimo proveedor supera el consumo durante entrega

delay_x_cpd = 800
min_qty = 1,000

Como $800 \leq 1,000$:

res_max = $800 + 1,000$
res_max = 1,800

Interpretación

Si el mínimo proveedor es mayor que el consumo operativo, el máximo se ajusta sumando ambos valores.

Cálculo de qty_req

```
CASE  
  WHEN qry.qty_req IS NULL THEN 0  
  ELSE qry.qty_req  
END AS qty_req
```

Qué significa

qty_req representa la cantidad requerida por nivelación.

Viene desde:

explode_lvl_bom.qty_req

Y originalmente viene de:

qty_level

Explicación funcional

Es la cantidad que falta para alcanzar el nivel objetivo configurado del producto, ya convertida a nivel materia prima si pasó por BOM.

Cálculo de **qty_sale**

```
CASE
  WHEN esd.qty IS NULL THEN 0
  ELSE esd.qty
END AS qty_sale
```

Fuente

```
LEFT JOIN explode_so_days('30 days'::interval) esd
  ON esd.product_id = qry.product_id
```

Qué significa

qty_sale representa la cantidad relacionada con venta abierta o requerimiento de ventas próximas, calculada por la función **explode_so_days** para un horizonte de 30 días.

Explicación funcional

Este campo mide demanda abierta futura de pedidos de venta dentro de los próximos 30 días, transformada a nivel producto o materia prima.

Cálculo de **need_purchased**

```
false AS need_purchased
```

Qué significa

En la vista SQL se inicializa siempre como falso.

La lógica real de necesidad de compra se calcula después en el modelo Python mediante la función **_comp_qty**.

Cálculo de **x_msg_exp**

`max(ppp.x_msg_exp) AS x_msg_exp`

Qué significa

Este campo toma un mensaje o valor adicional del producto desde:

`product_product.x_msg_exp`

No participa en las fórmulas principales de reabastecimiento, pero puede ser un campo informativo o de control.

Explicación textual para negocio

La vista **stock_warehouse_orderpoint_report** es la vista final de base de datos que arma el reporte de reglas de reabastecimiento.

Primero toma la información de consumo que ya fue calculada y explosionada por BOM: venta real, cantidad esperada, consumo total y consumo promedio diario. Después cruza esa información con el proveedor del producto para obtener los días de entrega y la cantidad mínima de compra.

Con esos datos calcula cuánto se consume durante el tiempo de entrega, el mínimo y el máximo sugerido. También agrega la cantidad requerida por nivelación y la demanda futura de pedidos de venta próximos.

Esta vista deja lista la información base que Odoo muestra en el reporte y que posteriormente complementa con cálculos Python de disponibilidad, tránsito y sugerencia de compra.

Relación principal de cálculos

`sum_qty = sale_qty + expc_qty`

`prom_qty = sum_qty / 90`

`delay_x_cpd = delay * prom_qty`

```
res_min =  
  si producto es consigna:  
    delay_x_cpd  
  si no:  
    delay_x_cpd * 2  
  
res_max =  
  si delay_x_cpd > min_qty:  
    delay_x_cpd * 2  
  si no:  
    delay_x_cpd + min_qty
```

Frase clave para memorizar

`stock_warehouse_orderpoint_report` toma el consumo promedio diario y lo combina con datos de proveedor para calcular mínimos, máximos y requerimientos de compra.

En una frase:

Consumo diario × días de entrega = base para calcular mínimo y máximo de reabastecimiento.